

Deep Learning and Causal AI for Estimation

Lecture 1: Feedforward Neural Networks

Stephen Hansen
University College London



FUNDACIÓN
RAMÓN ARECES



Center for
International
Finance

Introduction

Neural network models have driven many of the important breakthroughs in machine learning and AI over the last ten years.

They were already well-known and popular in the 1990s.

Several interrelated drivers of their recent growth:

1. Availability of large datasets for training
2. Hardware advances: GPU/TPU, cloud infrastructure
3. Software advances: high-level frameworks for model estimation that hide many details from the user
4. Algorithmic advances: better training algorithms

In this lecture, we describe the structure of the simplest class of neural networks: feed-forward neural networks.

Review of Supervised Learning

Begin with observations $(y_i, \mathbf{x}_i)_{i=1}^N$ where $\mathbf{x}_i$ contains J covariates.

Function $f_\theta(\mathbf{x})$ (potentially non-linear) that maps inputs into outputs.

θ is a vector of parameters that determines the shape of the function.

Typical workflow is

1. Split data into training T and test E portions
2. Fit $f_\theta(\mathbf{x})$ on training data to minimize loss function

$$\theta^* = \arg \min_{\theta} \sum_{i \in T} L(f_\theta(\mathbf{x}_i), y_i)$$

3. Evaluate test-set loss

$$\sum_{i \in E} L(f_{\theta^*}(\mathbf{x}_i), y_i)$$

Outline

Basic Architecture of a Neural Network

Estimation

A neuron

As in linear regression, we can linearly combine covariates and add a constant:

$$z_i = \theta_0 + \sum_{j=1}^J \theta_j x_{i,j}.$$

In neural network jargon, the $\{\theta_j\}_{j=1}^J$ terms are sometimes called *weights* and θ_0 is called a *bias*.

We can transform the linear combination via an *activation function* ϕ .

Activation Functions

Traditionally:

1. A sigmoid function (logistic regression):

$$\phi(z_i) = \frac{1}{1 + e^{-z_i}}$$

2. A particular limiting case: step function.
3. Hyperbolic tangent.
4. Identity function (linear regression): $\phi(z_i) = z_i$.

Other activation functions have gained popularity recently:

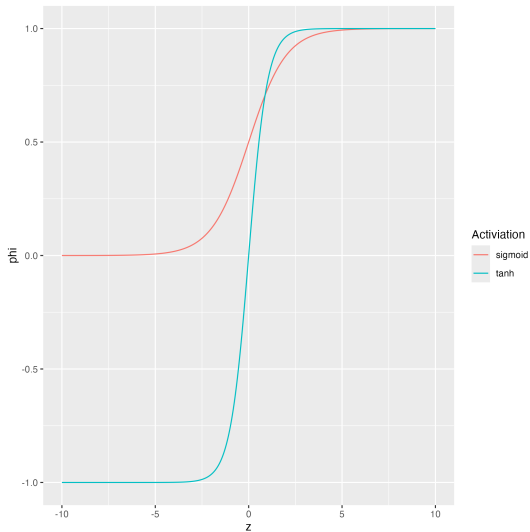
1. Rectified linear unit (ReLU):

$$\phi(z_i) = \max(0, z_i)$$

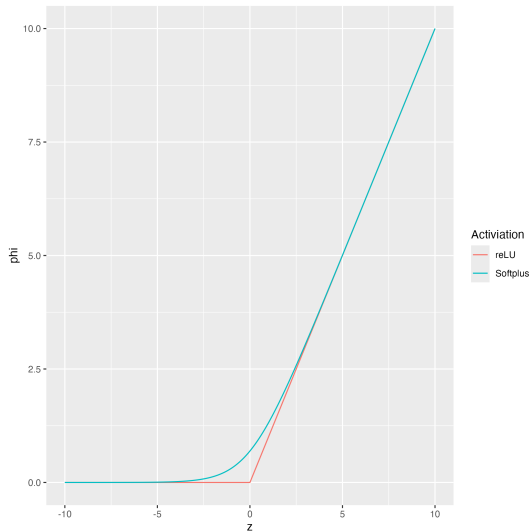
2. Softplus:

$$\phi(z_i) = \log(1 + e^{z_i})$$

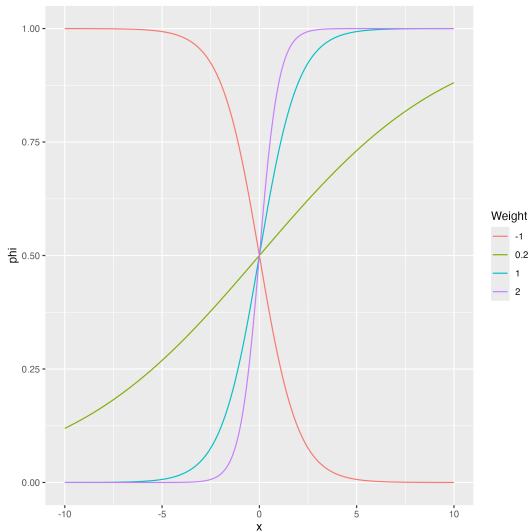
Activation Function Plots I



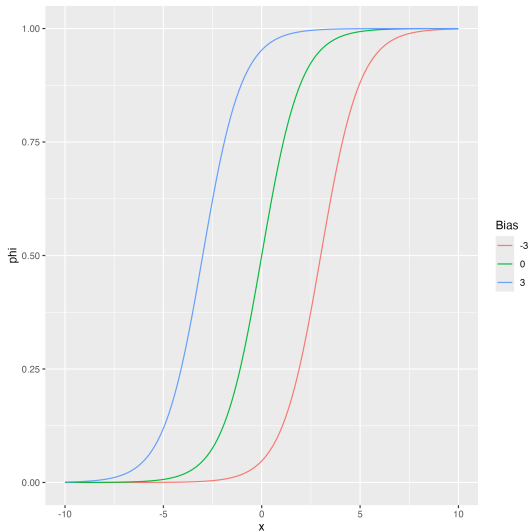
Activation Function Plots II



Effect of Varying Weight (Sigmoid)



Effect of Varying Bias (Sigmoid)



Combining Neurons into a Neural Network

Now suppose we have K separate activation functions operating over the same input $\mathbf{x}_i$.

We first obtain K separate linear combinations of covariates

$$z_{i,k} = \theta_{0,k} + \sum_{j=1}^J \theta_{j,k} x_{i,j} \quad \forall k = 1, \dots, K \quad (\text{Hidden Layer})$$

Next we linearly combine each separate neuron to create a final output

$$f_{\theta}(\mathbf{x}_i) = \theta_0 + \sum_k \theta_k \phi(z_{i,k}) \quad (\text{Output Layer})$$

This is known as a single layer network.

Consists of K non-linear transformations of linear combinations of input variables.

Matrix Notation

We can write the single-layer network compactly in matrix form.

Stack the weights into a $K \times J$ matrix $\mathbf{W}$ and the biases into $\mathbf{b} \in \mathbb{R}^K$:

$$\mathbf{h}_i = \phi(\mathbf{W}\mathbf{x}_i + \mathbf{b}) \quad (\text{Hidden Layer})$$

$$f_\theta(\mathbf{x}_i) = \mathbf{w}^\top \mathbf{h}_i + b_0 \quad (\text{Output Layer})$$

where ϕ is applied element-wise and $\mathbf{w} \in \mathbb{R}^K$, $b_0 \in \mathbb{R}$ are the output layer parameters.

Classic Result I

Result

Universal approximation theorem: [Hornik et al., 1989] A neural network with at least one hidden layer can approximate any Borel measurable function mapping finite-dimensional spaces to any desired degree of accuracy.

Deep Learning

The universal approximation theorem doesn't guarantee that a single layer suffices in practice.

An alternative to making a network wider is to make it *deeper*: stack multiple hidden layers so that each layer builds on features learned by the previous one.

In practice, adding layers provides a much more efficient parameterization than adding neurons to a single layer.

See [Montúfar et al., 2014] and [Telgarsky, 2016] for more technical discussion.

Adding Layers

Denote the output of layer l as $\mathbf{h}_i^{(l)}$, with $\mathbf{h}_i^{(0)} = \mathbf{x}_i$:

$$\mathbf{h}_i^{(1)} = \phi\left(\mathbf{W}^{(1)}\mathbf{x}_i + \mathbf{b}^{(1)}\right) \quad (\text{Hidden Layer 1})$$

$$\mathbf{h}_i^{(2)} = \phi\left(\mathbf{W}^{(2)}\mathbf{h}_i^{(1)} + \mathbf{b}^{(2)}\right) \quad (\text{Hidden Layer 2})$$

$$\vdots$$

$$f_{\theta}(\mathbf{x}_i) = \mathbf{w}^{(L+1)\top} \mathbf{h}_i^{(L)} + b^{(L+1)} \quad (\text{Output Layer})$$

where $\mathbf{W}^{(l)}$ is a $K_l \times K_{l-1}$ weight matrix, $\mathbf{b}^{(l)} \in \mathbb{R}^{K_l}$ is a bias vector, and ϕ is applied element-wise. For the first hidden layer, $K_0 = J$ since its input is $\mathbf{x}_i \in \mathbb{R}^J$.

Overall structure is called a *feed-forward neural network*.

Multiple layers allow for complex downstream structures to develop by combining simpler upstream structures.

Multi-Valued Output Layer

More generally, the output layer can produce M outputs:

$$\mathbf{f}_{\theta}(\mathbf{x}_i) = \mathbf{W}^{(L+1)}\mathbf{h}_i^{(L)} + \mathbf{b}^{(L+1)} \in \mathbb{R}^M$$

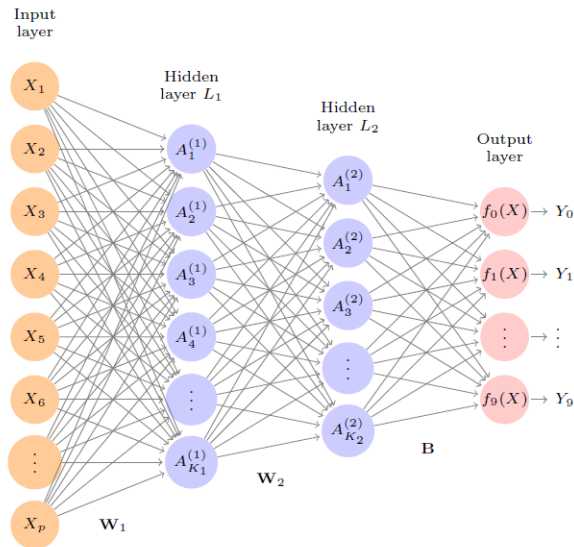
Two common uses:

1. Real-valued outputs: each f_m predicts a different target variable.
2. Class probabilities via the softmax function:

$$P_m(\mathbf{x}_i) = \frac{\exp(f_m)}{\sum_{m'=1}^M \exp(f_{m'})}$$

where M is the number of classes to predict.

Graphical Representation



Outline

Basic Architecture of a Neural Network

Estimation

Loss Functions and Gradient Descent

Recall that we want to find $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$ where $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(\mathbf{x}_i), y_i)$.

Typical choices for L :

1. Regression (mean squared error): $L = (y_i - f_{\theta}(\mathbf{x}_i))^2$
2. Classification (cross-entropy): $L = - \sum_{m=1}^M y_{i,m} \log P_m(\mathbf{x}_i \mid \theta)$

Gradient descent: repeatedly step in the direction of steepest descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

where $\eta > 0$ is the *learning rate* (a small step size).

Stochastic Gradient Descent

Computing $\nabla_{\theta} \mathcal{L}(\theta)$ requires summing over all N data points, which is expensive for large datasets.

Mini-batch SGD: instead approximate the gradient using a random batch $\mathcal{B}$ of B observations:

$$\theta \leftarrow \theta - \frac{\eta}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} L(f_{\theta}(\mathbf{x}_i), y_i)$$

This is an unbiased estimate of the full gradient, but much cheaper to compute.

An *epoch* is a single pass through all of the training data.

Automatic Differentiation

SGD requires computing $\nabla_{\theta}\mathcal{L}$, the gradient of the loss with respect to all network parameters.

Since a neural network is a composition of differentiable operations, the chain rule gives an exact gradient. *Automatic differentiation* is the algorithm that evaluates this efficiently.

Modern frameworks (JAX, PyTorch) make this a single function call.

References I

Hornik, K., Stinchcombe, M., and White, H. (1989).

Multilayer feedforward networks are universal approximators.

Neural Networks, 2(5):359–366.

Montúfar, G., Pascanu, R., Cho, K., and Bengio, Y. (2014).

On the number of linear regions of deep neural networks.

In Proceedings of the 28th International Conference on

Neural Information Processing Systems - Volume 2, volume 2 of NIPS'14, pages 2924–2932, Cambridge, MA, USA. MIT Press.

Telgarsky, M. (2016).

Benefits of depth in neural networks.

In Conference on Learning Theory, pages 1517–1539. PMLR.